FinCore

Secure Digital Wallet & Payments REST API

Phase 1 Explained

What was built, why it was built that way, and how it works
— in simple terms

Author	Piyush Kumar Sharma, B.Tech ECE, NIT Silchar
Repository	`https://github.com/piyush-nfx/fincore`
Commit	`8d0a2bd` on `main` (old CLI kept as tag `v1-cli`)
Stack	Java 25, Spring Boot 4.1, Spring Data JPA, MySQL 8
Status	Phase 1 done: 11/11 unit tests, 16/16 Postman checks

How to read this: every section follows the same pattern — **What** we built, **Why** we chose it, and **How** it works. Coloured boxes give plain-language summaries, warnings, and interview tips.

Contents

1 The Big Picture **3**

1.1 What is FinCore? . . . 3

1.2 Why rebuild CryptoVault? . . . 3

1.3 The four phases . . . 4

2 Setting Up the Project **4**

2.1 What we found on your machine . . . 4

2.2 The generated project, and two fixes . . . 4

2.3 The dependencies, one by one . . . 5

3 Configuration and Secrets **5**

3.1 What: `application.properties` . . . 5

3.2 Why secrets live in `.env` . . . 6

3.3 Why `open-in-view=false` . . . 6

3.4 The `jdbc_learning` incident (and the lesson) . . . 6

4 Architecture: How the Code Is Organised **6**

4.1 Layered architecture . . . 6

4.2 Package structure . . . 7

4.3 The journey of one request . . . 7

5 The Data Model (Entities) **8**

5.1 The three tables . . . 8

5.2 Design choices shared by all entities . . . 9

5.3 `User` . . . 9

5.4 `Wallet`: the most important entity . . . 9

5.5 `Transaction`: the immutable ledger . . . 10

6 Repositories: Talking to the Database **11**

7 DTOs: What Goes In and Out of the API **11**

7.1 What and why . . . 11

7.2 How: Java records . . . 12

7.3 Validation annotations . . . 12

8 Services: The Business Logic **12**

8.1 `@Transactional`, explained simply . . . 12

8.2 `UserService.register` . . . 13

8.3 `WalletService` . . . 13
8.4 `TransferService`: the heart of the project . . . 14
8.5 Idempotency: never send money twice . . . 15
8.6 `FailedTransferRecorder` and `REQUIRES_NEW` . . . 16
8.7 Concurrency and optimistic locking . . . 16

9 Controllers: The REST Endpoints **17**
9.1 The endpoints . . . 17
9.2 How a controller looks . . . 17

10 Error Handling **18**
10.1 Custom exceptions . . . 18
10.2 `GlobalExceptionHandler` . . . 19

11 Security in Phase 1 **19**

12 Testing **20**
12.1 Unit tests (11 tests, all passing) . . . 20
12.2 End-to-end testing with Postman (16/16 checks) . . . 21
12.3 Checking the database directly . . . 21

13 Git and the Project History **21**
13.1 What we did . . . 21
13.2 A small bug found in the old repo . . . 22
13.3 Other practical notes . . . 22

14 Known Limitations (Planned Fixes) **22**

15 Interview Questions You Can Now Answer **23**

16 Glossary **24**

1 The Big Picture

1.1 What is FinCore?

FinCore is the *backend* of a digital wallet, like the server behind Paytm or PhonePe (there is no app or website, only the server). Other programs talk to it over HTTP by sending JSON. It can:

- register users,
- create wallets for them (each wallet holds money in one currency, e.g. INR),
- deposit money into a wallet,
- transfer money from one wallet to another (peer-to-peer, "P2P"),
- keep a permanent record (a *ledger*) of every money movement.

The name is **Fin**ance + **Core**, from "core banking system": the central software of a bank that holds accounts and moves money.

1.2 Why rebuild CryptoVault?

FinCore replaces your older project, **CryptoVault**: a command-line Java program that simulated a wallet using an `ArrayList` in memory and saved data to a text file. That design is fine for learning OOP, but it breaks down for a real money system:

Problem	CryptoVault (v1)	FinCore (v2)
Many users at once	One person at a keyboard; an `ArrayList` is not safe if two threads change it together.	A web server handles many requests in parallel; the database + optimistic locking keep balances correct.
Durability	If the program crashes mid-write, the text file can be half-written or lost.	MySQL transactions: a change is either fully saved or not saved at all.
Who are you?	No login; whoever runs the program controls everything.	Users with BCrypt-hashed passwords now; JWT login and ownership checks in Phase 2.
History	A log file that can be edited.	An append-only ledger table that the code never updates.
Access	Only on your machine.	An HTTP API any client can call; deployable to the cloud (Phase 4).

In simple words

CryptoVault was a notebook on your desk. FinCore is a bank vault with a guard, a camera, and a logbook nobody can erase.

1.3 The four phases

Phase	Goal	Status
1 — Core API	Users, wallets, deposits, safe transfers, error handling, tests	**Done**
2 — Security	Login, JWT tokens, "you can only touch your own wallet", admin role	Next
3 — Production features	Automatic retry on conflicts, mandatory idempotency key, paginated history, Swagger	Planned
4 — Ship it	Concurrency test, Docker, deploy online, README	Planned

This document covers **everything done up to the end of Phase 1**, including setup, design decisions, every class, the tests, the Git history, and the problems we hit on the way.

2 Setting Up the Project

2.1 What we found on your machine

Before writing code, I checked which tools were installed, so the plan would match reality:

Tool	Found?	What it means
Java (JDK)	Yes, 25.0.3	We can use the newest Java.
Maven	No	Maven builds Java projects. We use the *Maven Wrapper* (`mvnw.cmd`) instead, which downloads the right Maven automatically.
MySQL	Yes, service `MySQL80` running	The database is ready locally.
Git	Yes	Version control works.
Docker	No	Needed only in Phase 4; you will install Docker Desktop then.

2.2 The generated project, and two fixes

You generated the starter project on `https://start.spring.io`. When I opened it I found two things that did not match the plan:

1. **Wrong database driver.** The `pom.xml` contained `mssql-jdbc`, which is the driver for *Microsoft SQL Server*, not MySQL. The app would never have connected to your database. I replaced it with `com.mysql:mysql-connector-j`.
2. **Spring Boot 4.1.2 instead of 3.x.** Your brief said 3.x, but the generator picked 4.1.2. We decided to **stay on 4.1**.

Why stay on Spring Boot 4.1?

Free (open-source) support for Spring Boot 3.5 ended in mid-2026, so 4.x is the line that still gets security fixes. For this project the code is almost identical. The differences you will see compared with 3.x tutorials are small: the web starter is called `spring-boot-starter-webmvc`, it uses Spring Framework 7 / Hibernate 7 / Jackson 3, and HTTP status 422 is called `UNPROCESSABLE_CONTENT`. Saying "I used the current supported version and handled the migra-

tion differences" is a good interview point.

2.3 The dependencies, one by one

A *dependency* is a library your project uses. Maven downloads them based on `pom.xml`.

Dependency	What it gives us
`spring-boot-starter-webmvc`	An embedded web server (Tomcat) and the tools to write REST controllers. JSON conversion via Jackson.
`spring-boot-starter-data-jpa`	JPA/Hibernate: map Java classes to tables, and Spring Data repositories so we do not write SQL for basic queries.
`mysql-connector-j`	The driver that lets Java talk to MySQL.
`spring-boot-starter-security`	Spring Security. In Phase 1 we use only its BCrypt password hasher and open all endpoints; Phase 2 locks them down.
`spring-boot-starter-validation`	Annotations like `@NotNull`, `@Positive`, `@Email` that check incoming data automatically.
`lombok`	Generates boilerplate (getters, constructors) at compile time, e.g. `@Getter`.
`spring-boot-devtools`	Auto-restarts the app when you change code (development only).
*-test starters	JUnit 5, Mockito, AssertJ and Spring test helpers.

3 Configuration and Secrets

3.1 What: `application.properties`

This file tells Spring Boot how to run. Here is the final version, explained line by line:

```
spring.application.name=fincore
server.port=8083

spring.config.import=optional:file:.env[.properties]
spring.datasource.url=${DB_URL:jdbc:mysql://localhost:3306/fincore?createDatabaseIfNotExist=true}
spring.datasource.username=${DB_USERNAME:root}
spring.datasource.password=${DB_PASSWORD}

spring.jpa.hibernate.ddl-auto=update
spring.jpa.open-in-view=false
spring.jpa.show-sql=false
```

- `server.port=8083` — you chose this port; the app listens on `http://localhost:8083`.
- `spring.config.import=optional:file:.env[.properties]` — also read settings from a file named `.env` in the project folder. `optional:` means "don't crash if it's missing".
- `${DB_URL:...}` — use the value of `DB_URL`; if it isn't set, use the default after the colon.
- `${DB_PASSWORD}` — no default on purpose: the password must come from `.env` or the environment.
- `ddl-auto=update` — Hibernate creates/adjusts tables to match the entity classes at startup. Handy while developing; in production we switch to `validate` (check, never change).

- `open-in-view=false` — explained below.

3.2 Why secrets live in `.env`

At one point you had written the real MySQL password directly into `application.properties`. That file is committed to GitHub, and **anything pushed to a public repository can be copied by anyone, even if you delete it later** (it stays in Git history). So:

- the real values went into `.env` (project root), which is listed in `.gitignore`, so Git never sees it;
- `.env.example` is committed instead, with placeholder values, so others know what to fill in;
- before committing, I searched the staged files for the password to confirm it was not included.

In simple words

`application.properties` is the recipe and is public. `.env` is the key to your house: it stays in your pocket. On the cloud server (Phase 4) the same names (`DB_PASSWORD`, etc.) will be set as environment variables in the hosting dashboard.

3.3 Why `open-in-view=false`

By default Spring keeps the database session open until the HTTP response is fully written ("Open Session In View"). That lets lazy-loaded data be fetched by accident while JSON is being generated: hidden extra queries and database connections held for too long. Turning it off forces all database work to happen inside the service layer's transactions, which is cleaner and easier to reason about. That is also why we convert entities to DTOs *inside* services.

3.4 The `jdbc\_learning` incident (and the lesson)

Your `.env` first pointed to your existing database `jdbc_learning`. It already had a `users` table (from your JDBC practice), with an `address` table referencing it. Because `ddl-auto=update` tries to make tables match the entities, Hibernate would *add* FinCore's columns to your old `users` table. When you tried to drop that table, MySQL refused (error 3730) because `address.uid` is a foreign key pointing at it.

Fix: FinCore got its own database. `DB_URL` now ends in `/fincore?createDatabaseIfNotExist=true`, so MySQL creates `fincore` on first start.

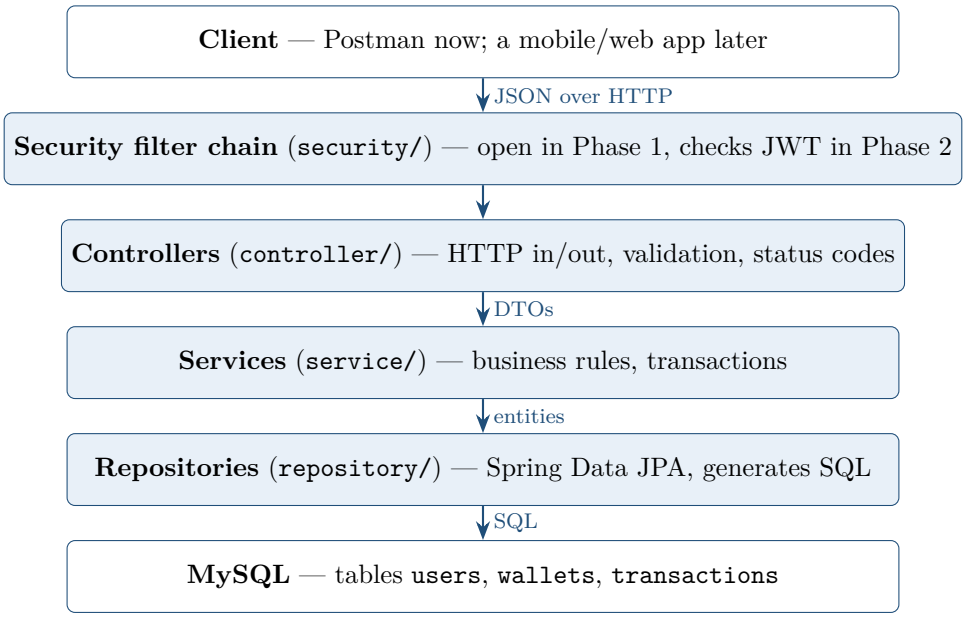
Lesson

One application, one database (or schema). Sharing a database between unrelated projects causes name clashes (two different `users` tables) and makes it risky to change either one.

4 Architecture: How the Code Is Organised

4.1 Layered architecture

The code is split into layers. Each layer has one job and only talks to the layer below it.



Alongside these layers: `entity/` (the Java classes that map to tables), `dto/` (the shapes of requests and responses), and `exception/` (custom errors plus one global error handler).

Why?

Separation of concerns. Controllers don't know SQL; repositories don't know HTTP. You can change one layer (e.g. swap MySQL for PostgreSQL, or add a GraphQL API) without rewriting the rest. It also makes testing easy: we unit-tested `TransferService` by replacing the repositories with fakes (mocks).

4.2 Package structure

```
com.piyush.fincore
|-- FincoreApplication.java          (main method, starts Spring Boot)
|-- controller   AuthController, WalletController, TransferController
|-- service      UserService, WalletService, TransferService, FailedTransferRecorder
|-- repository   UserRepository, WalletRepository, TransactionRepository
|-- entity       User, Wallet, Transaction, Role, TransactionType, TransactionStatus
|-- dto          RegisterRequest, CreateWalletRequest, DepositRequest, TransferRequest,
|                UserResponse, WalletResponse, TransactionResponse, ErrorResponse
|-- security     SecurityConfig
`-- exception    BusinessException + 5 subclasses, GlobalExceptionHandler
```

4.3 The journey of one request

Here is what happens, step by step, when Postman sends a transfer:

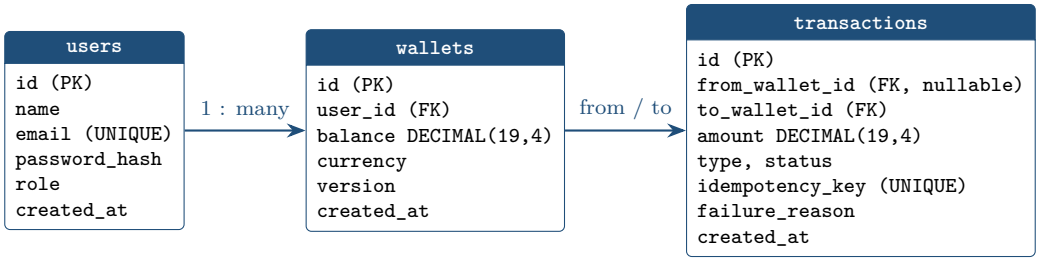
```
POST http://localhost:8083/api/transfers
Idempotency-Key: 6f1c...e2
{ "fromWalletId": 1, "toWalletId": 2, "amount": 250.50 }
```

1. **Tomcat** (the embedded web server) receives the HTTP request.
2. **Spring Security's filter chain** looks at it. In Phase 1 it lets everything through.

3. **DispatcherServlet** (Spring's front desk) finds the method mapped to `POST /api/transfers`: `TransferController.transfer()`.
4. **Jackson** converts the JSON body into a `TransferRequest` Java object.
5. **Bean Validation** checks the annotations (`@NotNull`, `@Positive`, …). If any fail, the request stops here with a 400 error.
6. The controller checks the `Idempotency-Key` header length and calls `TransferService.transfer()`.
7. Spring has wrapped the service in a **proxy** that **opens a database transaction**.
8. The service loads both wallets (SQL `SELECT`), debits, credits, and inserts a ledger row.
9. The method returns, so the proxy **commits**: Hibernate sends the `UPDATE`s and `INSERT` to MySQL, all at once.
10. The service returns a `TransactionResponse`; Jackson turns it into JSON; the client gets `201 Created`.
11. If *anything* threw an exception in steps 8–9, the proxy **rolls back** instead, and the `GlobalExceptionHandler` turns the exception into a clean JSON error.

5 The Data Model (Entities)

5.1 The three tables



- One **user** can own many **wallets** (e.g. an INR wallet and a USD wallet).
- Every **transaction** points to the wallet money went *to*, and (for transfers) the wallet it came *from*.

5.2 Design choices shared by all entities

Choice	Why
`@GeneratedValue(strategy = IDENTITY)`	MySQL's `AUTO_INCREMENT` assigns the id. Simple and standard for MySQL.
`@NoArgsConstructor(access = PROTECTED)`	JPA needs an empty constructor to build objects from rows, but our own code should use the real constructor, so it's hidden.
`@Getter` but **no** `@Setter`	Nobody can do `wallet.setBalance(1000000)`. State only changes through methods that enforce rules.
`@Enumerated(EnumType.STRING)`	Stores `"USER"` rather than `0`. If someone reorders the enum later, old data still means the same thing.
`@CreationTimestamp` on `created_at`	Hibernate fills in the time automatically on insert; `updatable = false` means it never changes.
`Instant` for time	A point on the global timeline (UTC), so no time-zone confusion between server and client.

5.3 User

Stores name, email (unique), the **BCrypt hash** of the password (never the password itself), and a role (`USER` or `ADMIN`). The role is used in Phase 2 for the admin endpoint.

5.4 Wallet: the most important entity

```
@Column(nullable = false, precision = 19, scale = 4)
private BigDecimal balance;

@Version
private Long version;

public void debit(BigDecimal amount) {
    requirePositive(amount);
    if (balance.compareTo(amount) < 0) {
        throw new InsufficientBalanceException(id, balance, amount);
    }
    balance = balance.subtract(amount);
}

public void credit(BigDecimal amount) {
    requirePositive(amount);
    balance = balance.add(amount);
}
```

Why `BigDecimal` and never `double`

Computers store `double` in binary, and most decimal fractions (like 0.1) cannot be written exactly in binary, just as 1/3 cannot be written exactly in decimal (0.333…). So:

```
0.1 + 0.2           // double     -> 0.30000000000000004
new BigDecimal("0.1").add(new BigDecimal("0.2"))   // -> 0.3 exactly
```

A few paise lost per transaction, times millions of transactions, is real money and an audit failure. `BigDecimal` stores numbers in base 10 exactly. In MySQL the column is `DECIMAL(19,4)`: up to 19

digits in total, 4 of them after the decimal point. Our unit test `decimalAmountsStayExact` proves this.

Comparing BigDecimals

Use `compareTo`, not `equals`. For `BigDecimal`, `equals` also compares the scale, so `new BigDecimal("2.0").equals(new BigDecimal("2.00"))` is **false**. That's why the code says `balance.compareTo(amount) < 0`.

Why `debit()`/`credit()` instead of `setBalance()`

This is called a **rich domain model**: the object protects its own rules. It is *impossible* anywhere in the code to make a balance negative or to add a negative amount, because the only way to change the balance is through these two methods, which check first. It also makes the rules trivially unit-testable (see `WalletTest`).

What `@Version` does (optimistic locking)

Every wallet row has a `version` number. Hibernate increases it on every update, and writes the update like this:

```
UPDATE wallets SET balance = 749.50, version = 3
WHERE id = 1 AND version = 2;
```

If another transaction already changed the wallet (so its version is now 3, not 2), the `WHERE` matches **0 rows**. Hibernate notices and throws an exception instead of silently overwriting the other change. Section 8.7 shows this with a timeline.

5.5 `Transaction`: the immutable ledger

```
@Entity
@Table(name = "transactions", indexes = { ... })
@Immutable                    // Hibernate never issues an UPDATE for this entity
public class Transaction {
    @ManyToOne(fetch = FetchType.LAZY)
    @JoinColumn(name = "from_wallet_id", updatable = false)
    private Wallet fromWallet;      // null for deposits
    ...
    @Column(name = "idempotency_key", unique = true, updatable = false, length = 64)
    private String idempotencyKey;

    public static Transaction deposit(Wallet to, BigDecimal amount) { ... }
    public static Transaction transfer(Wallet from, Wallet to, BigDecimal amount, String key) {
    ... }
    public static Transaction failedTransfer(Wallet from, Wallet to, BigDecimal amount,
                                             String key, String reason) { ... }
}
```

- **Immutable (append-only).** Rows are inserted, never changed or deleted. Every column is `updatable = false` and the class is marked `@Immutable`. Real banks work the same way: you never "edit" a past transaction; you add a new correcting one. This gives a full audit trail: you can always recompute a wallet's balance by replaying its ledger.
- **Static factory methods** (`deposit`, `transfer`, `failedTransfer`) instead of a public constructor with seven parameters. Each name says exactly what kind of row you're creating,

and it's impossible to build an invalid combination (e.g. a DEPOSIT with a from-wallet).

- `type` (DEPOSIT / TRANSFER) was added beyond the original brief so that deposits are recorded in the ledger too. Money entering the system is just as important to track as money moving inside it.
- `failure_reason` stores why a transfer failed (e.g. insufficient balance).
- **Indexes** on `from_wallet_id` and `to_wallet_id` make "show me this wallet's history" fast (needed for Phase 3's history endpoint).
- `FetchType.LAZY` means loading a transaction does not automatically load both wallets. They are fetched only if used, which avoids unnecessary queries.

6 Repositories: Talking to the Database

```
public interface TransactionRepository extends JpaRepository<Transaction, Long> {
    Optional<Transaction> findByIdempotencyKey(String idempotencyKey);
}
```

What: three interfaces, with no implementation written by us.

How: Spring Data JPA generates the implementation at startup. Extending `JpaRepository<Entity, IdType>` gives `save`, `findById`, `findAll`, `delete`, etc. for free. Methods named like `findByIdempotencyKey` or `existsByEmail` are *derived queries*: Spring reads the method name and builds the SQL (`SELECT ... WHERE idempotency_key = ?`).

Why: less code means fewer bugs. We only write custom queries when we need something special (Phase 3 will add one to avoid the N+1 problem when listing history).

In simple words

A repository is like a librarian: you ask "find the wallet with id 5" and it knows which shelf (table) to look on. You never touch the shelves yourself.

7 DTOs: What Goes In and Out of the API

7.1 What and why

DTO = *Data Transfer Object*: a simple class describing the exact shape of a request or response. We **never** return entities directly from controllers.

Why not just return the entity?

1. **Security:** returning `User` would leak `passwordHash` in the JSON.
2. **Stability:** the database can change (rename a column) without breaking the API for clients.
3. **No accidental queries:** serialising an entity can trigger lazy loading of related data, or even infinite loops (wallet → user → wallets → …).
4. **Mass assignment protection:** a client can't sneak `"balance": 1000000` into a request, because `CreateWalletRequest` simply has no such field.

7.2 How: Java records

```
public record TransferRequest(
        @NotNull Long fromWalletId,
        @NotNull Long toWalletId,
        @NotNull @Positive @Digits(integer = 15, fraction = 4) BigDecimal amount
) { }
```

A `record` is a compact, immutable data class: Java generates the constructor, getters (`request.amount()`), `equals`, `hashCode` and `toString`. Ideal for DTOs.

Response DTOs have a static `from(entity)` method that copies just the safe fields, e.g. `WalletResponse.from(wallet)`.

7.3 Validation annotations

Annotation	Meaning
`@NotNull`	Field must be present.
`@NotBlank`	String must contain at least one non-space character.
`@Positive`	Number must be > 0 (so no negative or zero transfers).
`@Digits(integer=15, fraction=4)`	At most 15 digits before the point and 4 after — matches `DECIMAL(19,4)` so nothing gets silently rounded.
`@Email`	Must look like an email address.
`@Size(min=8, max=72)`	Password length. Max 72 because BCrypt ignores anything after 72 bytes.
`@Pattern("[A-Z]{3}")`	Currency must be a 3-letter code like `INR`.

The controller marks the body with `@Valid`; if any rule fails, Spring throws `MethodArgumentNotValidException` *before* our code runs, and the global handler returns a 400 listing each bad field.

Interview angle

"Validation happens at the edge (DTOs), and the domain double-checks critical rules (`Wallet.debit` rejects non-positive amounts too). Defence in depth: even if some future code path skips the DTO, the entity still protects itself."

8 Services: The Business Logic

8.1 `@Transactional`, explained simply

A **database transaction** groups several SQL statements into one all-or-nothing unit. It guarantees the **ACID** properties:

Atomicity	All steps happen, or none do. No "debited but not credited".
Consistency	Constraints (unique email, foreign keys, NOT NULL) are always respected.
Isolation	Concurrent transactions don't see each other's half-finished work. (MySQL InnoDB default level: REPEATABLE READ.)
Durability	Once committed, the data survives a crash or power cut.

How Spring does it: when a class has `@Transactional` methods, Spring doesn't give other classes the real object; it gives them a **proxy** (a wrapper). When you call the method, the proxy:

1. opens a transaction,
2. calls the real method,
3. **commits** if it returns normally, or **rolls back** if it throws a `RuntimeException` (unchecked exceptions; checked exceptions do *not* roll back by default).

All our custom exceptions extend `RuntimeException` (via `BusinessException`), so they roll back.

8.2 UserService.register

```
@Transactional
public UserResponse register(RegisterRequest request) {
    String email = request.email().trim().toLowerCase(Locale.ROOT);
    if (userRepository.existsByEmail(email)) {
        throw new EmailAlreadyExistsException(email);
    }
    User user = new User(request.name().trim(), email,
                         passwordEncoder.encode(request.password()), Role.USER);
    return UserResponse.from(userRepository.save(user));
}
```

- **Normalise the email** (trim spaces, lower-case) so `Alice@X.com` and `alice@x.com` are the same account.
- **Check for duplicates** to give a friendly 409 error. The `UNIQUE` index on the column is the real guarantee: if two sign-ups race past the check at the same moment, the database rejects the second, and the handler still returns 409.
- **Hash with BCrypt**, never store the password. Every new user is a plain `USER`; nobody can register themselves as `ADMIN`.

BCrypt in simple words

Hashing is a one-way blender: you can turn a password into a hash, but not the hash back into the password. To log in later we blend the typed password again and compare. BCrypt also adds a random *salt* (so two users with the same password get different hashes) and is deliberately *slow* (around 100 ms), so an attacker who steals the table can't try billions of guesses per second. You saw this in Postman: registering took about 118 ms.

8.3 WalletService

```
@Transactional
public WalletResponse deposit(Long walletId, DepositRequest request) {
    Wallet wallet = findWallet(walletId);
    wallet.credit(request.amount());
    transactionRepository.save(Transaction.deposit(wallet, request.amount()));
    return WalletResponse.from(wallet);
}
```

- **create** — finds the user (404 if missing) and saves a new wallet with balance 0.

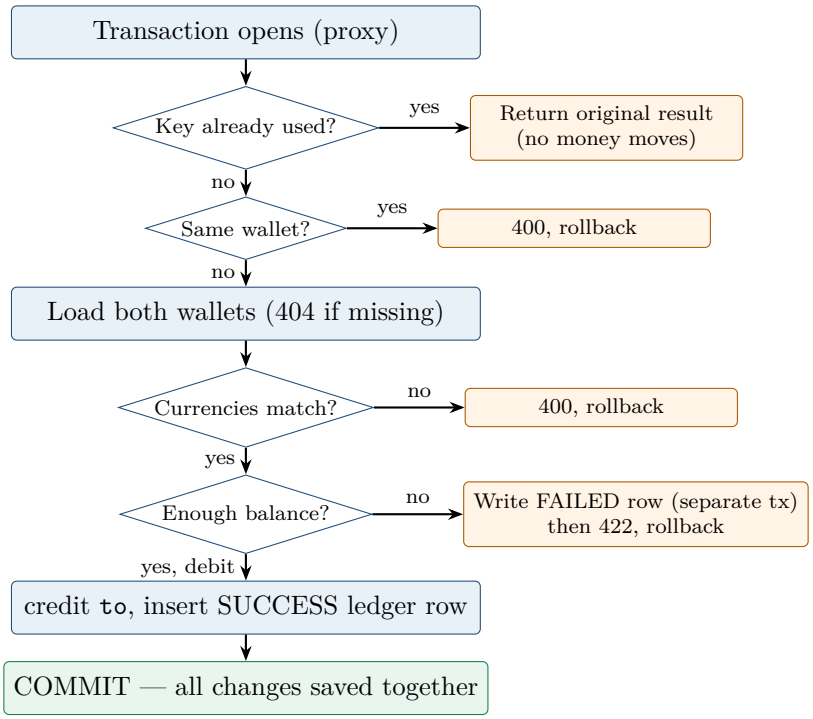
- **get** — marked `@Transactional(readOnly = true)`: a hint that lets Hibernate skip change-tracking work, and makes the intent clear.
- **deposit** — credits the wallet *and* writes a ledger row in the same transaction.

Why is there no `walletRepository.save(wallet)` in deposit?

Inside a transaction, objects loaded by Hibernate are **managed**: Hibernate remembers what they looked like when loaded. At commit it compares ("**dirty checking**") and automatically writes an `UPDATE` for anything that changed, including the version check. Calling `save()` again would be redundant.

8.4 `TransferService`: the heart of the project

```
@Transactional
public TransactionResponse transfer(TransferRequest request, String idempotencyKey) {
    // 1. Already processed this key? Return the original result.
    if (idempotencyKey != null) {
        Optional<Transaction> existing = transactionRepository.findByIdempotencyKey(idempotencyKey
    );
        if (existing.isPresent()) {
            return TransactionResponse.from(existing.get());
        }
    }
    // 2. Business rules
    if (request.fromWalletId().equals(request.toWalletId())) {
        throw new InvalidOperationException("Cannot transfer to the same wallet");
    }
    Wallet from = findWallet(request.fromWalletId());
    Wallet to = findWallet(request.toWalletId());
    if (!from.getCurrency().equals(to.getCurrency())) {
        throw new InvalidOperationException("Currency mismatch: ...");
    }
    // 3. Move the money
    try {
        from.debit(request.amount());
    } catch (InsufficientBalanceException e) {
        failedTransferRecorder.record(from.getId(), to.getId(), request.amount(),
                                      idempotencyKey, e.getMessage());
        throw e;
    }
    to.credit(request.amount());
    // 4. Record it in the ledger
    Transaction tx = transactionRepository.save(
            Transaction.transfer(from, to, request.amount(), idempotencyKey));
    return TransactionResponse.from(tx);
}
```



- **Cheap checks first.** The same-wallet check needs no database, so it runs before loading anything (our unit test verifies the repository is never touched in that case).
- **Everything in one transaction.** If the ledger insert failed after the debit, the debit is undone too. Money can never vanish or be created.
- **Currency check.** Sending 100 INR into a USD wallet as "100 USD" would create money out of thin air.

8.5 Idempotency: never send money twice

The problem

You tap "Pay" and your internet drops. Did the payment go through? The app doesn't know, so it retries. Without protection, the server might process *both* requests: you pay twice.

The solution: the client generates a unique ID (a UUID) per payment attempt and sends it in the `Idempotency-Key` header. Retries reuse the same key. The server stores the key with the ledger row:

- first request with key `K`: transfer happens, row saved with `idempotency_key = K`;
- retry with key `K`: the lookup finds the existing row and returns it. **No second transfer.**

Postman request 7 proved this: same key, same transaction id returned, Alice charged once (749.50).

What if two identical requests arrive at the exact same moment? Both may pass the lookup. But the `UNIQUE` constraint on `idempotency_key` means only one `INSERT` can succeed. The other transaction fails at commit and **rolls back entirely, including its debit**. So money still moves once. (In Phase 1 the loser gets a 409; Phase 3 will make it return the original result instead.)

Interview angle

"Idempotent means doing it twice has the same effect as doing it once. The check-then-act lookup is the fast path; the unique constraint is the real guarantee, because only the database can arbitrate a race between two concurrent requests."

8.6 FailedTransferRecorder and REQUIRES_NEW

```
@Component
public class FailedTransferRecorder {
    @Transactional(propagation = Propagation.REQUIRES_NEW)
    public void record(Long fromWalletId, Long toWalletId, BigDecimal amount,
                       String idempotencyKey, String reason) {
        transactionRepository.save(Transaction.failedTransfer(
                walletRepository.getReferenceById(fromWalletId),
                walletRepository.getReferenceById(toWalletId),
                amount, idempotencyKey, reason));
    }
}
```

What: when a transfer fails for insufficient balance, we still want a `FAILED` row in the ledger (useful for auditing and fraud detection).

The trap: the transfer's transaction is about to roll back. If we inserted the FAILED row inside it, the rollback would delete that row too. Nothing would be recorded.

The fix: `Propagation.REQUIRES_NEW` tells Spring to *pause* the current transaction, start a brand-new independent one, commit it, then resume the original (which then rolls back). The FAILED row survives.

Why a separate class? Remember the proxy: `@Transactional` only works when the call goes *through* the proxy. If `TransferService` called its own `this.record()`, the call would skip the proxy ("self-invocation"), `REQUIRES_NEW` would be ignored, and the row would be rolled back. Putting it in another bean forces the call through a proxy.

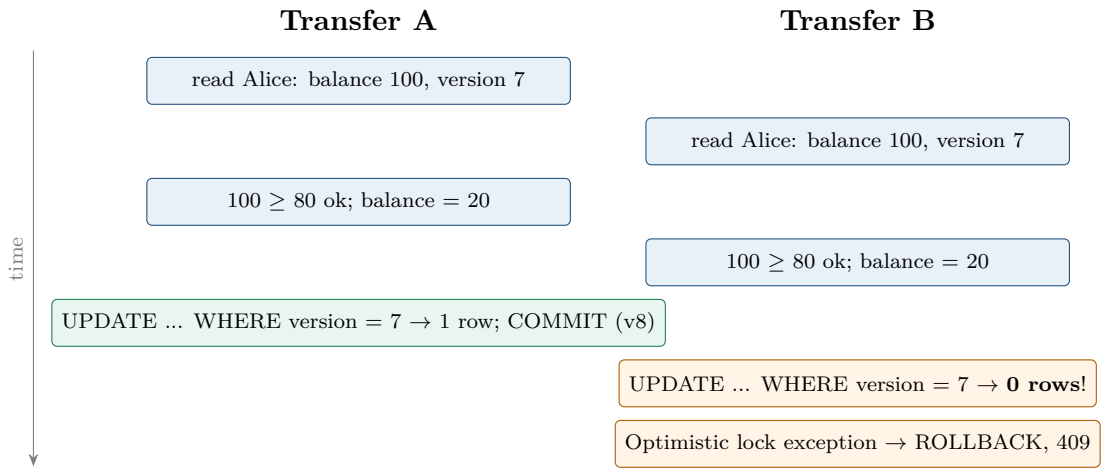
`getReferenceById` returns a lightweight placeholder holding only the id. That's all we need to fill in the foreign keys, so no extra `SELECT` runs.

Interview angle

This is one of the best questions to steer an interview toward: "How do you log a failure when the transaction that failed rolls back?" Answer: `REQUIRES_NEW` in a separate bean, and explain self-invocation. It shows you understand how Spring works inside, not just how to use it.

8.7 Concurrency and optimistic locking

The danger ("lost update" / double spending): Alice has 100. Two transfers of 80 arrive at the same time on two threads.



Without `@Version`, B's update would simply overwrite A's: Alice ends with 20 but 160 was sent. That's **80 rupees created from nothing**. With `@Version`, B's stale update matches no rows, so it fails safely. Alice ends at 20, only 80 was sent, and the books balance.

	Optimistic locking (what we use)	Pessimistic locking (alternative)
Idea	Don't lock; detect conflicts at write time with a version number.	Lock the row when reading (`SELECT ... FOR UPDATE`); others wait.
Good when	Conflicts are rare (most transfers touch different wallets).	Conflicts are frequent (one very hot account).
Cost	Loser must retry.	Waiting, and risk of deadlocks if locks are taken in different orders.

In Phase 1 the losing request gets **409 Conflict** ("please retry"). Phase 3 adds automatic retries *outside* the transaction, and Phase 4 proves it with a test firing 10 concurrent transfers.

9 Controllers: The REST Endpoints

9.1 The endpoints

Method	**Path**	What it does	Success
POST	`/api/auth/register`	Create a user (password hashed)	201
POST	`/api/wallets`	Create a wallet for a user	201 + `Location`
GET	`/api/wallets/{id}`	Get a wallet and its balance	200
POST	`/api/wallets/{id}/deposit`	Add money to a wallet	200
POST	`/api/transfers`	Send money between wallets	201

9.2 How a controller looks

```
@RestController
@RequestMapping("/api/wallets")
@RequiredArgsConstructor
public class WalletController {
    private final WalletService walletService;

    @PostMapping
```

```
    public ResponseEntity<WalletResponse> create(@Valid @RequestBody CreateWalletRequest request)
    {
        WalletResponse wallet = walletService.create(request);
        URI location = ServletUriComponentsBuilder.fromCurrentRequest()
                .path("/{id}").buildAndExpand(wallet.id()).toUri();
        return ResponseEntity.created(location).body(wallet);
    }
    ...
}
```

- `@RestController` = this class handles HTTP and its return values become JSON.
- `@RequiredArgsConstructor` (Lombok) generates a constructor for the `final` fields; Spring uses it to **inject** the service (*constructor injection*, the recommended style: dependencies are explicit and the class is easy to test).
- `@RequestBody` = read the JSON body; `@PathVariable` = read `\{id\}` from the URL; `@RequestHeader` = read a header (the idempotency key).
- Creating something returns **201 Created** plus a `Location` header pointing to the new resource, which is standard REST practice.

Why are controllers so thin?

Controllers only translate HTTP to Java and back. All rules live in services, so the same logic could be reused from a message queue or a scheduled job, and tested without starting a web server.

Temporary shortcut: `CreateWalletRequest` contains a `userId`, so anyone could create a wallet for anyone. That's acceptable only because Phase 1 has no login. In Phase 2 the owner will come from the JWT token, and `userId` will be removed from the request.

10 Error Handling

10.1 Custom exceptions

```
public abstract class BusinessException extends RuntimeException {
    private final HttpStatus status;
    protected BusinessException(HttpStatus status, String message) { ... }
}
public class WalletNotFoundException extends BusinessException {
    public WalletNotFoundException(Long id) {
        super(HttpStatus.NOT_FOUND, "Wallet " + id + " not found");
    }
}
```

Exception	HTTP	When
`WalletNotFoundException`	404	Wallet id doesn't exist
`UserNotFoundException`	404	User id doesn't exist (creating a wallet)
`InsufficientBalanceException`	422	Not enough money
`InvalidOperationException`	400	Same-wallet transfer, currency mismatch, bad amount, bad idempotency key
`EmailAlreadyExistsException`	409	Email already registered

Why a base class that carries the status? The global handler needs only *one* method for all business errors. Adding a new error type later means writing one small class, not touching the handler.

Why 422 for insufficient balance? The request is perfectly valid JSON (so not 400); the server understood it but can't carry it out given the current state. 422 "Unprocessable Content" says exactly that.

10.2 `GlobalExceptionHandler`

A class annotated `@RestControllerAdvice` catches exceptions thrown by any controller and turns them into the same JSON shape:

```
{
  "timestamp": "2026-11-01T12:30:05Z",
  "status": 422,
  "error": "Unprocessable Content",
  "message": "Insufficient balance in wallet 1: available 749.5000, requested 999999",
  "path": "/api/transfers"
}
```

Caught	HTTP	Why
`BusinessException (all subclasses)`	own	Our expected errors
`MethodArgumentNotValidException`	400	`@Valid` failed; response lists `fieldErrors`
`HttpMessageNotReadableException`	400	Broken JSON
`MethodArgumentTypeMismatchException`	400	e.g. `/api/wallets/abc`
`ObjectOptimisticLockingFailureException`	409	`@Version` conflict — safe to retry
`DataIntegrityViolationException`	409	A unique constraint fired (race)
`Exception (anything else)`	500	Logged on the server; client sees only "Unexpected error"

Why?

Consistency: clients need one parser for all errors. **Security:** stack traces reveal class names, library versions and SQL, all useful to attackers. Unknown errors are logged in full on the server but the client only sees a generic message.

11 Security in Phase 1

Adding the Spring Security dependency *locks every endpoint by default* (it generates a random password and demands login). Since login only arrives in Phase 2, `SecurityConfig` temporarily opens everything:

```
@Bean
public SecurityFilterChain securityFilterChain(HttpSecurity http) throws Exception {
    http.csrf(AbstractHttpConfigurer::disable)
        .sessionManagement(s -> s.sessionCreationPolicy(SessionCreationPolicy.STATELESS))
        .authorizeHttpRequests(auth -> auth.anyRequest().permitAll());
    return http.build();
}

@Bean
public PasswordEncoder passwordEncoder() { return new BCryptPasswordEncoder(); }
```

- **CSRF disabled:** CSRF attacks abuse *cookies* that browsers send automatically. Our API will use tokens in the `Authorization` header, which browsers never attach on their own, so CSRF protection isn't needed.
- **STATELESS:** the server keeps no login session in memory; every request must prove who it is (with a JWT in Phase 2). This lets you run many server copies behind a load balancer.
- **PasswordEncoder bean:** one shared BCrypt hasher, injected into `UserService`.

Phase 1 is NOT secure yet — on purpose

Right now anyone can read any wallet or move money from any wallet. That's acceptable only on your laptop while building. Phase 2 fixes it with JWT authentication and ownership checks, and the app will not be deployed publicly before that.

12 Testing

12.1 Unit tests (11 tests, all passing)

A **unit test** checks one class in isolation, in milliseconds, without a database or web server.

`WalletTest` (5 tests): the money rules

- `creditThenDebitUpdatesBalance` — 100.50 − 40.25 = 60.25 exactly.
- `decimalAmountsStayExact` — 0.1 + 0.2 = 0.3 (would fail with `double`).
- `debitMoreThanBalanceIsRejectedAndBalanceUnchanged` — even 0.0001 too much is rejected and nothing changes.
- `debitOfExactBalanceLeavesZero` — spending everything is allowed.
- `zeroOrNegativeAmountsAreRejected`.

`TransferServiceTest` (6 tests): the transfer logic with Mockito

Mockito creates fake versions (*mocks*) of the repositories. We tell them what to return ("when `findById(1)` is called, return Alice's wallet") and later *verify* what was called. This tests the service's logic alone, with no MySQL needed.

```
@Test
void insufficientBalanceRecordsFailedRowAndChangesNothing() {
    stubWallets(alice, bob);                        // alice has 100
    assertThatThrownBy(() -> transferService.transfer(
            new TransferRequest(1L, 2L, new BigDecimal("100.01")), "key-2"))
        .isInstanceOf(InsufficientBalanceException.class);

    assertThat(alice.getBalance()).isEqualByComparingTo("100");   // unchanged
    verify(failedTransferRecorder).record(eq(1L), eq(2L), eq(new BigDecimal("100.01")),
                                          eq("key-2"), anyString());
    verify(transactionRepository, never()).save(any());          // no SUCCESS row
}
```

The six cases: successful transfer; insufficient balance (FAILED row recorded, nothing else); same-wallet rejected before touching the DB; unknown wallet → 404; currency mismatch; repeated idempotency key returns the original without moving money.

Tests follow the **Arrange–Act–Assert** pattern: set up data, call the method, check the result. Run them with:

```
.\mvnw.cmd test
```

(The generated `FincoreApplicationTests.contextLoads` starts the whole app, so it needs MySQL running and your `.env` in place.)

12.2 End-to-end testing with Postman (16/16 checks)

Unit tests use fakes; Postman tests the *real* running app against the *real* database. The collection `postman/FinCore-Phase1.postman_collection.json` runs 12 requests in order. Each request saves the ids the next ones need (e.g. Alice's wallet id) using small test scripts.

Request	Result	What it proves
1–2 Register Alice, Bob	201	Users created, passwords hashed
3–4 Create wallets	201	Wallets start at 0
5 Deposit 1000	200	Balance exactly 1000; DEPOSIT ledger row
6 Transfer 250.50	201	SUCCESS, both balances updated in one transaction
7 Replay same key	201	Same transaction id: **no double payment**
8 Check Alice	200	749.50: debited exactly once
9 Transfer 999999	422	Insufficient balance, FAILED row recorded
10 Same wallet	400	Business rule enforced
11 Amount −50	400	Validation, with `fieldErrors.amount`
12 Wallet 999999	404	Clean not-found error

The red 4xx badges in Postman are *expected*: those requests were designed to fail, and the tests check that they fail correctly.

12.3 Checking the database directly

```
SELECT id, balance, version FROM fincore.wallets;
SELECT id, type, status, amount, failure_reason FROM fincore.transactions;
```

You should see the `version` column increase each time a wallet changes, and a ledger with one DEPOSIT, one SUCCESS TRANSFER and one FAILED TRANSFER per collection run.

13 Git and the Project History

13.1 What we did

1. Your GitHub repo `piyush-nfx/fincore` was empty. Your CryptoVault code was in a separate local repo (`Desktop/Projects/cryovault`, remote `piyush-nfx/CryptoVault`) with 3 commits.
2. In the FinCore folder: `git init -b main` and added the GitHub remote.
3. Imported CryptoVault's history without touching the original folder:

```
git fetch ../../cryovault main      # copy its commits into this repo
git reset --soft FETCH_HEAD         # point main at them, keep FinCore files staged
```

```
git tag -a v1-cli -m "CryptoVault v1 ..." FETCH_HEAD
```

4. Committed FinCore on top. That commit adds the 45 Spring Boot files and removes the 16 old CLI files from `main`; the tag still holds them.
5. Verified the password was not in any staged file, then pushed `main` and the tag.

```
8d0a2bd (main)   FinCore v2: rebuild CryptoVault as a Spring Boot wallet REST API (Phase 1)
5536a58 (v1-cli) Feature: Added interactive CLI menu loop using Scanner
571c587          Initial commit: Complete OOP Crypto Wallet Simulation
784f0dc          Initial commit: Core OOP wallet architecture with history and file logging
```

Why keep the old history?

It tells your growth story in the repo itself: v1 CLI → v2 REST API. An interviewer can run `git checkout v1-cli` to see where you started. A **tag** is a permanent bookmark on a commit, ideal for marking a version.

13.2 A small bug found in the old repo

CryptoVault's ignore file was named `.gitignore.txt`. Git only reads a file named exactly `.gitignore`, so it was ignored and IntelliJ's `.idea` files got committed. FinCore has a proper `.gitignore` (which also excludes `target/` and `.env`).

13.3 Other practical notes

- The project lives inside **OneDrive**. Sync can lock files during builds. If you see strange "file in use" errors, move the project to a non-synced folder like `D:\textbackslash Projects`.
- The commit message includes a `Co-Authored-By` line because the code was written with an AI assistant. Be ready to explain every line yourself; this document is meant to help with exactly that.

14 Known Limitations (Planned Fixes)

Being honest about limitations is a strength in interviews. These are deliberate for Phase 1:

Limitation	Fix	Phase
No login: anyone can read or move anyone's money	JWT authentication + ownership checks	2
`userId` sent in the create-wallet body	Owner taken from the token	2
Optimistic-lock conflict returns 409; client must retry	Automatic retry (up to 3) outside the transaction	3
`Idempotency-Key` is optional	Make it required for transfers	3
Keys are unique globally, not per user	Scope keys to the user	3
Same-key race: loser gets 409	Catch the conflict and return the original result	3
Replaying a FAILED key still answers 201	Return the matching status code	3
No transaction history endpoint	`GET /api/transactions?page=0&size=10`	3
No concurrency proof yet	10-thread double-spend test	4

15 Interview Questions You Can Now Answer

Q: Why did you rebuild CryptoVault?

An `ArrayList` plus a text file can't handle concurrent users, isn't crash-safe, and has no authentication. I moved to Spring Boot + MySQL to get ACID transactions, optimistic locking, and (Phase 2) JWT security.

Q: Why BigDecimal for money?

`double` is binary floating point and can't represent 0.1 exactly ($0.1 + 0.2 = 0.30000000000000004$). `BigDecimal` is exact decimal arithmetic; the column is `DECIMAL(19,4)`. And compare with `compareTo`, not `equals`.

Q: How do you make sure a transfer is all-or-nothing?

Debit, credit and the ledger insert run in one `@Transactional` method. Spring's proxy commits if the method returns and rolls back on any `RuntimeException`, so a partial transfer can never be saved.

Q: What happens if two transfers hit the same wallet at once?

Both read version n. The first commits and bumps it to $n + 1$. The second's `UPDATE ... WHERE version = n` matches zero rows, Hibernate throws an optimistic-locking exception, the transaction rolls back, and the client gets 409. No lost update, no double spend.

Q: Optimistic vs pessimistic locking?

Optimistic detects conflicts at commit time (cheap when conflicts are rare; loser retries). Pessimistic locks rows with `SELECT ... FOR UPDATE` (others wait; better for hot rows; risk of deadlocks). Most transfers touch different wallets, so optimistic fits.

Q: What is idempotency and how did you implement it?

Repeating a request has the same effect as sending it once. Clients send an `Idempotency-Key`; if we've seen it, we return the stored result. A `UNIQUE` constraint on the column guarantees it even when two copies race.

Q: How do you record a failed transfer if the transaction rolls back?

A separate bean with `@Transactional(propagation = REQUIRES_NEW)` commits the FAILED row in its own transaction. It must be a separate bean because self-invocation bypasses Spring's proxy.

Q: Why DTOs instead of returning entities?

To avoid leaking fields like the password hash, to decouple the API from the database schema, to prevent lazy-loading surprises and infinite JSON loops, and to block mass assignment.

Q: Why is the ledger immutable?

Auditability. Like a bank statement, past entries are never edited; a correction is a new entry. Every balance can be recomputed from the ledger.

Q: Why BCrypt?

It's one-way, salted per password, and deliberately slow, so stolen hashes are expensive to crack. It only uses the first 72 bytes, so we cap password length at 72.

Q: What does `open-in-view=false` do?

It closes the persistence context when the service transaction ends, so no hidden lazy-loading queries run while JSON is being written, and DB connections are held for less time.

Q: What is dirty checking?

Hibernate tracks entities loaded in a transaction and, at commit, automatically issues `UPDATE`s for whatever changed. That's why deposit doesn't call `save(wallet)`.

Q: How did you keep secrets out of GitHub?

Config reads `${DB_PASSWORD}`; real values live in a git-ignored `.env` (imported via `spring.config.import`) locally, and in environment variables on the server. A committed `.env.example` documents the names.

16 Glossary

REST API A way for programs to talk over HTTP using URLs (`/api/wallets/1`) and verbs (GET, POST).

JSON Text format for data: `\{"amount": 250.50\}`.

Spring Boot Framework that wires your Java app together and runs an embedded web server.

Bean An object Spring creates and manages (services, repositories, controllers).

Dependency injection Spring hands a class the objects it needs (via the constructor) instead of the class creating them.

JPA / Hibernate JPA is the standard for mapping objects to tables; Hibernate is the implementation that writes the SQL.

Entity A Java class mapped to a table.

DTO A plain class/record describing API input or output.

Transaction A group of DB operations that succeed or fail together.

ACID Atomicity, Consistency, Isolation, Durability.

Proxy	A wrapper object Spring puts around a bean to add behaviour (like transactions) before/after method calls.
Optimistic locking	Detect concurrent edits using a version number instead of locking.
Idempotency	Doing something twice has the same effect as doing it once.
Ledger	Append-only list of every money movement.
BCrypt	Slow, salted, one-way password hashing algorithm.
Mock	A fake object used in tests to stand in for a real dependency.
HTTP status	2xx success, 4xx client mistake, 5xx server failure.
JWT	JSON Web Token, a signed token proving who you are (Phase 2).
Git tag	A permanent named bookmark on a commit (e.g. `v1-cli`).

Next: Phase 2 — Security. Login with JWT access and refresh tokens, a filter that verifies the token on every request, "you can only use your own wallets" (preventing IDOR), and an ADMIN-only endpoint.